

Live Streaming Platform — MVP

A Production-Grade Video Streaming System
with Design Patterns and SOLID Principles

Team Members:

Ranajeet Roy (Project Lead)
Narendra Singh
Pranav Singh
Keshav Rajput
Jivit Rana

Software Design & Software Engineering (SDSE)

Course Project Report

May 4, 2026

Abstract

This report presents the design and implementation of a production-grade live streaming platform similar to Twitch or YouTube Live. The system is architected using NestJS and TypeScript for the backend, React with Vite for the frontend, PostgreSQL via Prisma ORM for data persistence, and Nginx RTMP for streaming infrastructure. The project successfully demonstrates the practical application of seven Gang-of-Four design patterns (Singleton, Factory, Strategy, Observer, Adapter, Template Method, and Composite) and adheres to all five SOLID principles. The platform supports real-time video streaming via RTMP/HLS, WebSocket-based chat functionality, JWT authentication, subscription-based access control, and comprehensive API endpoints. This implementation showcases enterprise-level software engineering practices including layered architecture, dependency injection, input validation, and security best practices.

Contents

1	Introduction	7
1.1	Problem Statement	7
1.2	Objectives	7
1.3	Scope	7
2	Literature Review / Background	8
2.1	Live Streaming Architecture	8
2.2	Design Patterns Overview	8
2.3	SOLID Principles	9
2.4	Modern Web Technologies	9
3	System Architecture	9
3.1	High-Level Architecture	9
3.2	Component Diagram	11
3.3	Data Flow Architecture	12
3.4	Layered Architecture	13
4	Technology Stack	14
4.1	Backend Technologies	14
4.1.1	NestJS Framework	14
4.1.2	Prisma ORM v7	14
4.1.3	PostgreSQL Database	14
4.2	Frontend Technologies	15
4.2.1	React with Vite	15
4.2.2	HLS.js Library	15
4.3	Streaming Infrastructure	15
4.3.1	Nginx RTMP Module	15
4.4	Authentication & Security	16
4.4.1	JWT (JSON Web Tokens)	16
4.4.2	bcrypt Password Hashing	16
4.5	Real-Time Communication	16
4.5.1	Socket.io	16
5	Database Design	16
5.1	Entity-Relationship Model	16
5.2	Schema Details	17
5.2.1	User Model	17
5.2.2	Channel Model	18
5.2.3	Stream Model	18
5.2.4	Subscription Model	19
5.2.5	Comment Model	20
5.3	Database Configuration	20

6	Design Patterns Implementation	21
6.1	Singleton Pattern	21
6.1.1	Pattern Description	21
6.1.2	Implementation: PrismaService	21
6.1.3	Class Diagram	22
6.1.4	Why This Pattern Was Chosen	22
6.2	Factory Pattern	22
6.2.1	Pattern Description	22
6.2.2	Implementation: StreamFactory	22
6.2.3	Class Diagram	23
6.2.4	Why This Pattern Was Chosen	24
6.3	Strategy Pattern	24
6.3.1	Pattern Description	24
6.3.2	Implementation: Viewing Strategies	24
6.3.3	Class Diagram	26
6.3.4	Why This Pattern Was Chosen	26
6.4	Observer Pattern	27
6.4.1	Pattern Description	27
6.4.2	Implementation: Stream Events	27
6.4.3	Class Diagram	28
6.4.4	Why This Pattern Was Chosen	29
6.5	Adapter Pattern	29
6.5.1	Pattern Description	29
6.5.2	Implementation: Streaming Adapter	29
6.5.3	Class Diagram	30
6.5.4	Why This Pattern Was Chosen	30
6.6	Template Method Pattern	31
6.6.1	Pattern Description	31
6.6.2	Implementation: BaseService	31
6.6.3	Class Diagram	32
6.6.4	Why This Pattern Was Chosen	33
6.7	Composite Pattern	33
6.7.1	Pattern Description	33
6.7.2	Implementation: Channel Hierarchy	33
6.7.3	Class Diagram	35
6.7.4	Why This Pattern Was Chosen	35
7	SOLID Principles Analysis	36
7.1	Single Responsibility Principle (SRP)	36
7.1.1	Evidence in Architecture	36
7.1.2	Additional Examples	37
7.2	Open/Closed Principle (OCP)	37
7.2.1	Evidence: Strategy Pattern	37
7.2.2	Evidence: Observer Pattern	38
7.2.3	Evidence: Template Method	38
7.3	Liskov Substitution Principle (LSP)	39
7.3.1	Evidence: BaseService Substitutability	39
7.3.2	Evidence: Strategy Pattern	39

7.3.3	Evidence: Composite Pattern	39
7.4	Interface Segregation Principle (ISP)	39
7.4.1	Evidence: Narrow Interfaces	40
7.4.2	Evidence: Separate DTOs	40
7.4.3	Evidence: Guards	40
7.5	Dependency Inversion Principle (DIP)	41
7.5.1	Evidence: NestJS Dependency Injection	41
7.5.2	Evidence: Repository Abstraction	41
7.5.3	Evidence: Adapter Pattern	41
8	API Design	42
8.1	RESTful Endpoints	42
8.2	WebSocket Events	43
8.3	Authentication Flow	44
8.4	Request/Response Examples	45
8.4.1	Register User	45
8.4.2	Get Live Streams	45
9	Frontend Architecture	45
9.1	Component Hierarchy	45
9.2	Routing	46
9.3	State Management	47
9.4	Video Player Integration	47
9.5	Real-Time Chat	48
10	Security Implementation	49
10.1	Authentication Mechanisms	49
10.1.1	Password Security	49
10.1.2	JWT Token Security	49
10.2	Authorization Mechanisms	50
10.2.1	JwtAuthGuard	50
10.2.2	SubscriptionGuard	50
10.3	Input Validation	50
10.4	RTMP Authentication	51
10.5	CORS Configuration	52
11	Streaming Pipeline	52
11.1	RTMP to HLS Conversion	52
11.2	Nginx Configuration	53
11.3	OBS Setup	54
11.4	HLS Playback	54
11.5	Latency Optimization	55
12	Testing & Deployment	55
12.1	Setup Instructions	55
12.1.1	Prerequisites	55
12.1.2	Backend Setup	55
12.1.3	Frontend Setup	56
12.1.4	Nginx Setup	56

12.2	Running the Application	56
12.3	Testing the Streaming Flow	57
12.4	Testing Patterns	58
12.4.1	Strategy Pattern Test	58
12.4.2	Observer Pattern Test	58
13	UML Diagram Explanations	58
13.1	Sequence Diagram (sequence.png)	58
13.2	Use Case Diagram (usecase.png)	59
13.3	Class Diagram (class.png)	61
13.4	Entity-Relationship Diagram (er.png)	62
14	Team Contribution Matrix	64
15	Conclusion	65
15.1	Summary	65
15.2	Achievements	65
15.3	Challenges Overcome	66
15.4	Future Improvements	66
15.5	Reflections	67
16	References	67

List of Figures

1	System Sequence Diagram - Complete Request Flow	10
2	Use Case Diagram - User Interactions and System Features	11
3	Entity-Relationship Diagram	17
4	Class Diagram - Complete System Architecture	36

List of Tables

1	Authentication Endpoints	42
2	Channel Endpoints	42
3	Stream Endpoints	43
4	Subscription Endpoints	43
5	Streaming Infrastructure Endpoints	43
6	WebSocket Events	44
7	Team Member Contributions	64

1 Introduction

1.1 Problem Statement

The proliferation of video content creation has led to an increasing demand for accessible, scalable live streaming platforms. Traditional streaming solutions often face challenges including:

- Complex infrastructure requirements for video ingestion and delivery
- Real-time interaction mechanisms between streamers and viewers
- Scalable architectures that can handle concurrent users
- Secure authentication and authorization systems
- Maintainable codebases that adhere to software engineering principles

This project addresses these challenges by building a minimum viable product (MVP) of a live streaming platform that demonstrates both functional capabilities and exemplary software design.

1.2 Objectives

The primary objectives of this project are:

1. Design and implement a fully functional live streaming platform supporting RTMP ingestion and HLS playback
2. Demonstrate practical application of seven Gang-of-Four design patterns in a real-world context
3. Implement all five SOLID principles throughout the codebase
4. Create a layered architecture that separates concerns and promotes maintainability
5. Integrate real-time communication via WebSocket for chat functionality
6. Implement secure authentication using JWT and bcrypt password hashing
7. Develop a subscription-based access control system with tiered viewing privileges
8. Document comprehensive API design following RESTful principles

1.3 Scope

This project encompasses:

- Backend API development using NestJS and TypeScript
- Frontend application using React, Vite, and HLS.js
- Database design and ORM integration using Prisma with PostgreSQL

- Streaming infrastructure configuration using Nginx RTMP module
- Real-time chat system using Socket.io
- Authentication and authorization mechanisms
- Subscription management with viewing strategy patterns
- Comprehensive testing and deployment documentation

The project specifically demonstrates software design patterns and SOLID principles while maintaining production-ready code quality.

2 Literature Review / Background

2.1 Live Streaming Architecture

Live streaming platforms have evolved significantly over the past decade. Modern architectures typically employ a separation between video ingestion (RTMP protocol) and delivery (HLS or DASH protocols). RTMP (Real-Time Messaging Protocol) was developed by Adobe for low-latency audio and video transmission, while HLS (HTTP Live Streaming) by Apple provides adaptive bitrate streaming over HTTP, making it compatible with standard web infrastructure [3].

The typical pipeline involves:

1. Video capture from OBS (Open Broadcaster Software) or similar encoders
2. RTMP transmission to a media server
3. Transcoding and segmentation into HLS format
4. Distribution via CDN or HTTP servers
5. Client-side playback using adaptive bitrate algorithms

2.2 Design Patterns Overview

Design patterns, as documented by the Gang of Four (Gamma et al.), provide reusable solutions to common software design problems [1]. The patterns implemented in this project are:

Creational Patterns:

- **Singleton:** Ensures a class has only one instance with global access
- **Factory:** Encapsulates object creation logic

Behavioral Patterns:

- **Strategy:** Defines a family of algorithms and makes them interchangeable
- **Observer:** Establishes one-to-many dependency between objects
- **Template Method:** Defines algorithm skeleton with customizable steps

Structural Patterns:

- **Adapter:** Converts one interface into another
- **Composite:** Composes objects into tree structures

2.3 SOLID Principles

The SOLID principles, introduced by Robert C. Martin, form the foundation of object-oriented design [2]:

- **Single Responsibility Principle (SRP):** A class should have only one reason to change
- **Open/Closed Principle (OCP):** Open for extension, closed for modification
- **Liskov Substitution Principle (LSP):** Subtypes must be substitutable for their base types
- **Interface Segregation Principle (ISP):** Clients should not depend on interfaces they don't use
- **Dependency Inversion Principle (DIP):** Depend on abstractions, not concretions

2.4 Modern Web Technologies

The project leverages modern web development technologies:

- **NestJS:** A progressive Node.js framework that uses TypeScript and implements design patterns natively through dependency injection and decorators
- **Prisma ORM:** A next-generation ORM that provides type safety and database abstractions
- **React with Vite:** Modern frontend framework with optimized build tooling
- **WebSocket (Socket.io):** Enables bidirectional real-time communication

3 System Architecture

3.1 High-Level Architecture

The Live Streaming Platform follows a three-tier architecture with clear separation of concerns:

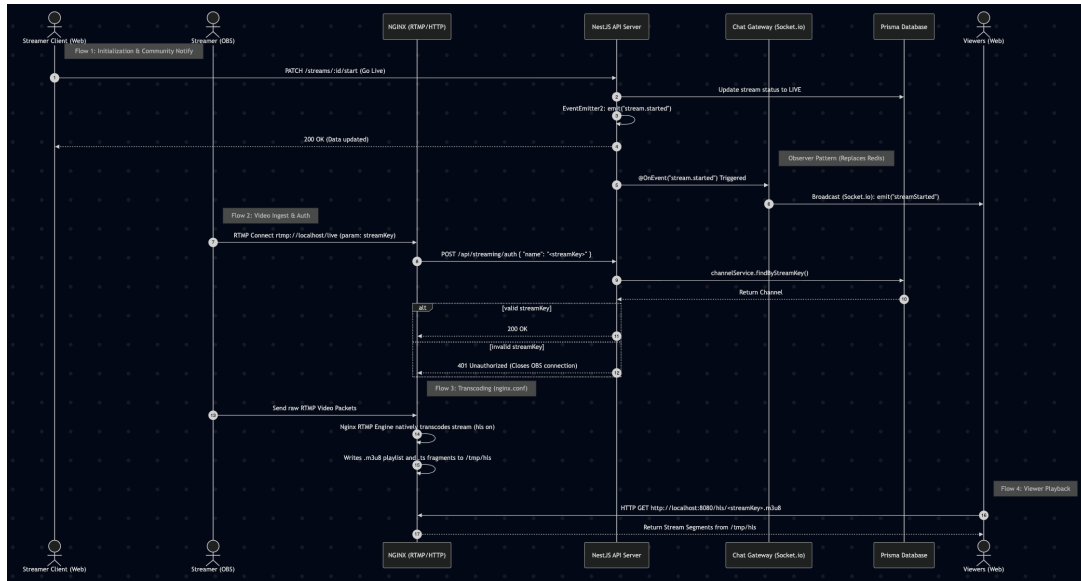


Figure 1: System Sequence Diagram - Complete Request Flow

The architecture consists of three primary tiers:

Presentation Tier (Frontend):

- React-based single-page application served on port 5173
- HLS.js library for adaptive video playback
- Socket.io client for real-time chat
- Axios for HTTP API communication with JWT interceptors

Application Tier (Backend):

- NestJS server on port 3001 handling business logic
- RESTful API with 16 endpoints
- WebSocket gateway for real-time events
- JWT authentication using Passport.js
- Layered architecture: Controllers → Services → Repositories

Data Tier:

- PostgreSQL database hosted on NeonDB
- Prisma ORM v7 with connection pooling
- Five core models with relationship mapping

Streaming Infrastructure:

- Nginx RTMP module receiving streams on port 1935
- HLS transcoding with 3-second segments
- HTTP server on port 8080 serving HLS playlists
- On-publish callback for stream key validation

3.2 Component Diagram

The system is organized into seven feature modules, each encapsulating specific domain logic:



Figure 2: Use Case Diagram - User Interactions and System Features

Core Modules:

1. **Auth Module:** User registration, login, JWT token generation
2. **User Module:** User profile management and CRUD operations
3. **Channel Module:** Streaming channel management, stream key generation
4. **Stream Module:** Stream lifecycle management (create, start, stop)
5. **Subscription Module:** Tiered access control with viewing strategies
6. **Chat Module:** WebSocket-based real-time messaging
7. **Streaming Module:** Integration adapter for Nginx RTMP

Each module follows the same internal structure:

Module

Controller (HTTP endpoints)
Service (Business logic)
Repository (Data access)
DTOs (Data transfer objects)
Entities/Interfaces

3.3 Data Flow Architecture

The system implements a unidirectional data flow:

Request Flow:

1. Client sends HTTP request to Controller
2. Controller validates request using ValidationPipe
3. JwtAuthGuard verifies authentication token
4. SubscriptionGuard checks access permissions (if required)
5. Controller delegates to Service layer
6. Service executes business logic, may call multiple repositories
7. Repository performs database operations via PrismaService
8. Response flows back through the layers

Streaming Flow:

1. Broadcaster starts OBS with RTMP URL and stream key
2. Nginx receives RTMP stream on port 1935
3. Nginx triggers on_publish callback to Backend
4. Backend validates stream key against database
5. If valid, Nginx accepts stream and begins HLS transcoding
6. HLS segments written to /tmp/hls/ directory
7. Nginx HTTP server serves .m3u8 playlist on port 8080
8. Frontend fetches playlist and streams via HLS.js

3.4 Layered Architecture

The backend implements a strict four-layer architecture:

- Controllers (HTTP Handlers)
 - Route definitions (@Get, @Post)
 - Request/Response transformation
 - Guard application

↓

- Services (Business Logic)
 - Domain logic implementation
 - Transaction orchestration
 - Pattern implementations

↓

- Repositories (Data Access)
 - Database queries
 - Prisma interaction
 - BaseRepository abstraction

↓

- PrismaService (Database Layer)
 - Connection management
 - Query execution
 - Transaction handling

Dependency Rules:

- Controllers never directly access PrismaService
- Services never handle HTTP concerns (Request/Response objects)
- Repositories never contain business logic
- Each layer only depends on the layer directly below it

This strict layering ensures:

- Testability: Each layer can be tested in isolation
- Maintainability: Changes in one layer don't cascade
- Flexibility: Layers can be swapped (e.g., changing database)
- Clarity: Clear separation of concerns

4 Technology Stack

4.1 Backend Technologies

4.1.1 NestJS Framework

Justification: NestJS was selected as the backend framework for several compelling reasons:

- **TypeScript Native:** Provides strong typing, reducing runtime errors
- **Built-in Dependency Injection:** Facilitates SOLID principles implementation
- **Modular Architecture:** Natural fit for feature-based organization
- **Decorator Pattern:** Simplifies metadata-driven development (guards, pipes, interceptors)
- **Express/Fastify Compatible:** Leverages mature HTTP ecosystem
- **WebSocket Support:** Integrated Socket.io support for real-time features

4.1.2 Prisma ORM v7

Justification: Prisma was chosen over traditional ORMs (TypeORM, Sequelize) due to:

- **Type Safety:** Auto-generated TypeScript types from schema
- **Declarative Schema:** Schema-first approach with migrations
- **Query Performance:** Optimized query engine
- **Prisma Client:** Intuitive, auto-completed query API
- **Database Compatibility:** Support for PostgreSQL, MySQL, SQLite
- **Version 7 Features:** Driver adapters for edge deployments, improved connection pooling

4.1.3 PostgreSQL Database

Justification: PostgreSQL was selected as the RDBMS for:

- **ACID Compliance:** Ensures data consistency for critical operations
- **Advanced Features:** JSON support, full-text search, materialized views
- **Scalability:** Handles concurrent connections efficiently
- **NeonDB Integration:** Serverless PostgreSQL with automatic scaling
- **Indexing Capabilities:** Supports complex queries on stream metadata

4.2 Frontend Technologies

4.2.1 React with Vite

Justification:

- **Component-Based Architecture:** Reusable UI components
- **Virtual DOM:** Efficient rendering for real-time updates
- **Vite Build Tool:** Lightning-fast HMR (Hot Module Replacement)
- **JSX Syntax:** Declarative UI composition
- **Rich Ecosystem:** Vast library support

4.2.2 HLS.js Library

Justification:

- **Browser Compatibility:** Works in browsers without native HLS support
- **Adaptive Bitrate:** Automatic quality adjustment
- **MSE Integration:** Uses Media Source Extensions API
- **Error Recovery:** Automatic retry mechanisms

4.3 Streaming Infrastructure

4.3.1 Nginx RTMP Module

Justification:

- **Mature Technology:** Battle-tested in production environments
- **High Performance:** C-based implementation, minimal overhead
- **HLS Transcoding:** Built-in support for HLS segment generation
- **Callback Hooks:** `on_publish`, `on_done` for stream lifecycle events
- **Low Latency:** 3-5 second glass-to-glass latency achievable
- **Open Source:** No licensing costs, community support

Configuration highlights:

```
1 rtmp {
2     server {
3         listen 1935;
4         application live {
5             live on;
6             hls on;
7             hls_path /tmp/hls;
8             hls_fragment 3s;
9             on_publish http://localhost:3001/api/streaming/auth;
10        }
11    }
12 }
```


4.4 Authentication & Security

4.4.1 JWT (JSON Web Tokens)

Justification:

- **Stateless Authentication:** No server-side session storage
- **Cross-Domain Support:** Suitable for microservices
- **Payload Encryption:** Secure user data transmission
- **Expiration Control:** 7-day token lifetime

4.4.2 bcrypt Password Hashing

Justification:

- **Salt Integration:** Automatic salt generation
- **Adaptive Function:** Configurable complexity (12 rounds)
- **Rainbow Table Resistance:** Salted hashes prevent precomputation attacks

4.5 Real-Time Communication

4.5.1 Socket.io

Justification:

- **Bidirectional Events:** Server and client can both initiate messages
- **Room Support:** Namespace-based chat room isolation
- **Fallback Mechanisms:** WebSocket with long-polling fallback
- **NestJS Integration:** @nestjs/websockets module

5 Database Design

5.1 Entity-Relationship Model

The database schema consists of five core entities with well-defined relationships:

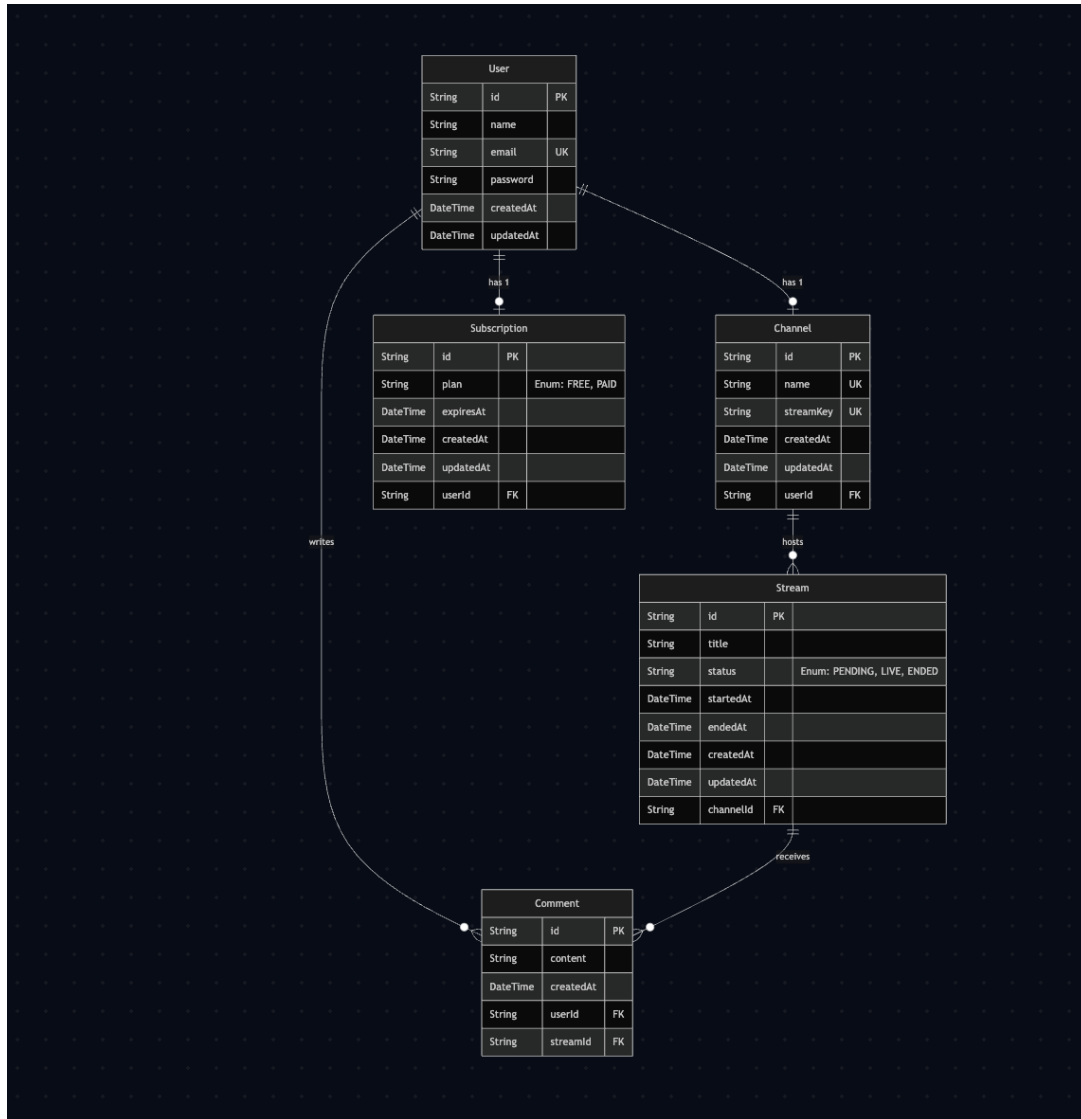


Figure 3: Entity-Relationship Diagram

Relationships:

- User → Channel: One-to-One (Each user has exactly one channel)
- User → Subscription: One-to-One (Each user has one subscription)
- User → Comment: One-to-Many (Users can post multiple comments)
- Channel → Stream: One-to-Many (Channels can have multiple streams)
- Stream → Comment: One-to-Many (Streams can have multiple comments)

5.2 Schema Details**5.2.1 User Model**

```

1 model User {
2   id          String          @id @default(cuid())

```

```

3  name      String
4  email      String      @unique
5  password   String      // bcrypt hashed
6  createdAt  DateTime     @default(now())
7  updatedAt  DateTime     @updatedAt
8
9  channel     Channel?
10 subscription Subscription?
11 comments    Comment []
12
13 @@index([email])
14 }

```

Design Decisions:

- `cuid()`: Collision-resistant unique identifier
- `@unique` on email: Prevents duplicate registrations
- `password`: Never exposed in API responses
- `updatedAt`: Automatic timestamp management

5.2.2 Channel Model

```

1  model Channel {
2    id      String      @id @default(cuid())
3    name     String      @unique
4    streamKey String      @unique @default(cuid())
5    userId   String      @unique
6    createdAt DateTime    @default(now())
7    updatedAt DateTime    @updatedAt
8
9    user      User      @relation(fields: [userId],
10                               references: [id],
11                               onDelete: Cascade)
12    streams   Stream []
13
14    @@index([streamKey])
15 }

```

Design Decisions:

- `streamKey`: Auto-generated secret for RTMP authentication
- `@unique` on name: Each channel has a unique URL slug
- `onDelete: Cascade`: Deleting user deletes channel
- `@@index` on `streamKey`: Fast lookup during RTMP validation

5.2.3 Stream Model

```

1  enum StreamStatus {
2      PENDING
3      LIVE
4      ENDED
5  }
6
7  model Stream {
8      id          String          @id @default(cuid())
9      title       String
10     status      StreamStatus @default(PENDING)
11     startedAt   DateTime?
12     endedAt     DateTime?
13     channelId    String
14     createdAt   DateTime         @default(now())
15     updatedAt   DateTime         @updatedAt
16
17     channel      Channel          @relation(fields: [channelId],
18                                         references: [id],
19                                         onDelete: Cascade)
20
21     comments     Comment []
22
23     @@index([channelId])
24     @@index([status])
25 }

```

Design Decisions:

- StreamStatus enum: Type-safe status transitions
- startedAt/endedAt: Nullable for PENDING streams
- @@index on status: Efficient live stream queries
- @@index on channelId: Fast channel stream lookups

5.2.4 Subscription Model

```

1  enum SubscriptionPlan {
2      FREE
3      PAID
4  }
5
6  model Subscription {
7      id          String          @id @default(cuid())
8      plan        SubscriptionPlan @default(FREE)
9      expiresAt   DateTime?
10     userId       String          @unique
11     createdAt    DateTime         @default(now())
12     updatedAt    DateTime         @updatedAt
13
14     user          User            @relation(fields: [userId],
15                                         references: [id],
16                                         onDelete: Cascade)
17
18     @@index([userId])
19 }

```

Design Decisions:

- `SubscriptionPlan` enum: Supports future tiers (PREMIUM, etc.)
- `expiresAt`: Nullable for FREE plan (never expires)
- Strategy pattern: Plan determines viewing behavior

5.2.5 Comment Model

```

1 model Comment {
2   id          String    @id @default(cuid())
3   content     String
4   userId      String
5   streamId    String
6   createdAt   DateTime @default(now())
7
8   user        User      @relation(fields: [userId],
9                             references: [id],
10                            onDelete: Cascade)
11  stream      Stream    @relation(fields: [streamId],
12                                references: [id],
13                                onDelete: Cascade)
14
15  @@index([streamId])
16  @@index([userId])
17 }

```

Design Decisions:

- Dual foreign keys: Links users and streams
- `@@index`: Optimizes chat retrieval per stream
- Cascade deletion: Cleanup when stream/user deleted

5.3 Database Configuration

Prisma v7 configuration with driver adapter:

```

1 // prisma.config.ts
2 import { PrismaClient } from '@prisma/client';
3 import { PrismaPg } from '@prisma/adapter-pg';
4 import { Pool } from 'pg';
5
6 const pool = new Pool({
7   connectionString: process.env.DATABASE_URL,
8   max: 20, // Connection pool size
9 });
10
11 const adapter = new PrismaPg(pool);
12 const prisma = new PrismaClient({ adapter });
13
14 export default prisma;

```

Configuration Highlights:

- **Connection Pooling:** Reuses database connections
- **Driver Adapter:** Enables edge runtime support
- **Environment Variables:** DATABASE_URL from .env
- **NeonDB Integration:** Serverless PostgreSQL

6 Design Patterns Implementation

This section provides detailed analysis of each implemented Gang-of-Four design pattern.

6.1 Singleton Pattern

6.1.1 Pattern Description

The Singleton pattern ensures that a class has only one instance and provides a global point of access to it. This is particularly useful for managing shared resources like database connections.

6.1.2 Implementation: PrismaService

```
1 // src/common/database/prisma.service.ts
2 import { Injectable, OnModuleInit, OnModuleDestroy } from '@nestjs/
  common';
3 import { PrismaClient } from '@prisma/client';
4
5 @Injectable()
6 export class PrismaService
7   extends PrismaClient
8   implements OnModuleInit, OnModuleDestroy {
9
10   async onModuleInit() {
11     await this.$connect();
12     console.log('Database connected');
13   }
14
15   async onModuleDestroy() {
16     await this.$disconnect();
17     console.log('Database disconnected');
18   }
19 }
```

```
1 // src/common/database/prisma.module.ts
2 import { Global, Module } from '@nestjs/common';
3 import { PrismaService } from '../prisma.service';
4
5 @Global()
6 @Module({
7   providers: [PrismaService],
8   exports: [PrismaService],
9 })
10 export class PrismaModule {}
```

6.1.3 Class Diagram

```
<<Injectable>>
  PrismaService

- extends PrismaClient

+ onModuleInit(): Promise
+ onModuleDestroy(): Promise
+ user: UserDelegate
+ channel: ChannelDelegate
+ stream: StreamDelegate
+ subscription: ...Delegate
+ comment: CommentDelegate

(singleton instance)

NestJS DI
Container
```

6.1.4 Why This Pattern Was Chosen

- **Resource Management:** Database connections are expensive; pooling must be centralized
- **Consistency:** All modules interact with the same Prisma instance
- **Lifecycle Management:** NestJS DI container ensures proper initialization/cleanup
- **Global Access:** @Global decorator makes it available across all modules
- **Thread Safety:** Node.js single-threaded model eliminates concurrency concerns

6.2 Factory Pattern

6.2.1 Pattern Description

The Factory pattern encapsulates object creation logic, allowing clients to create objects without specifying their exact classes. This promotes loose coupling between creation and usage.

6.2.2 Implementation: StreamFactory

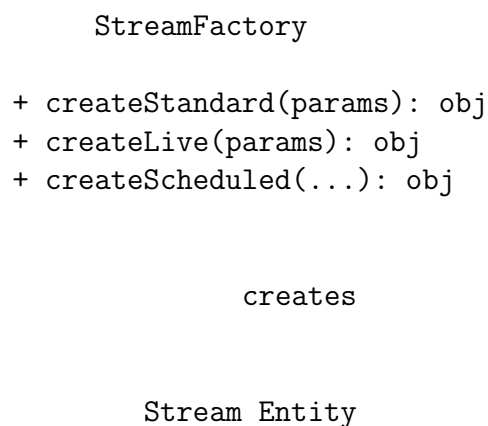
```
1 // src/modules/stream/stream.factory.ts
2 import { Injectable } from '@nestjs/common';
3 import { StreamStatus } from '@prisma/client';
4
```

```

5 export interface StreamCreationParams {
6   title: string;
7   channelId: string;
8 }
9
10 @Injectable()
11 export class StreamFactory {
12
13   createStandard(params: StreamCreationParams) {
14     return {
15       title: params.title,
16       channelId: params.channelId,
17       status: StreamStatus.PENDING,
18       startedAt: null,
19       endedAt: null,
20     };
21   }
22
23   createLive(params: StreamCreationParams) {
24     return {
25       title: params.title,
26       channelId: params.channelId,
27       status: StreamStatus.LIVE,
28       startedAt: new Date(),
29       endedAt: null,
30     };
31   }
32
33   createScheduled(params: StreamCreationParams,
34                   scheduledTime: Date) {
35     return {
36       title: params.title,
37       channelId: params.channelId,
38       status: StreamStatus.PENDING,
39       startedAt: scheduledTime,
40       endedAt: null,
41     };
42   }
43 }

```

6.2.3 Class Diagram



- title: string
- channelId: string
- status: StreamStatus
- startedAt: Date?
- endedAt: Date?

6.2.4 Why This Pattern Was Chosen

- **Encapsulation:** Hides complex initialization logic from controllers
- **Consistency:** Ensures all streams are created with valid state
- **Extensibility:** New stream types can be added without modifying existing code
- **Testability:** Factory can be mocked for unit tests
- **Clarity:** Client code clearly expresses intent (createLive vs createStandard)

6.3 Strategy Pattern

6.3.1 Pattern Description

The Strategy pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategies let the algorithm vary independently from clients that use it.

6.3.2 Implementation: Viewing Strategies

```
1 // src/modules/subscription/strategies/viewing-strategy.interface.ts
2 export interface ViewingResult {
3   canWatch: boolean;
4   maxDuration: number | null; // seconds, null = unlimited
5   reason?: string;
6 }
7
8 export interface ViewingStrategy {
9   canWatch(userId: string, streamId: string): ViewingResult;
10  getPlanName(): string;
11 }
```

```
1 // src/modules/subscription/strategies/free-viewing.strategy.ts
2 import { Injectable } from '@nestjs/common';
3 import { ViewingStrategy, ViewingResult } from '../viewing-strategy.interface';
4
5 @Injectable()
6 export class FreeViewingStrategy implements ViewingStrategy {
7
8   canWatch(userId: string, streamId: string): ViewingResult {
9     return {
10       canWatch: true,
11       maxDuration: 300, // 5 minutes

```

```
12     reason: 'Free tier: 5-minute viewing limit',
13   };
14 }
15
16 getPlanName(): string {
17   return 'FREE';
18 }
19 }
```

```
1 // src/modules/subscription/strategies/paid-viewing.strategy.ts
2 import { Injectable } from '@nestjs/common';
3 import { ViewingStrategy, ViewingResult } from '../viewing-strategy.
4   interface';
5
6 @Injectable()
7 export class PaidViewingStrategy implements ViewingStrategy {
8
9   canWatch(userId: string, streamId: string): ViewingResult {
10     return {
11       canWatch: true,
12       maxDuration: null, // Unlimited
13       reason: 'Paid tier: unlimited viewing',
14     };
15   }
16
17   getPlanName(): string {
18     return 'PAID';
19   }
20 }
```

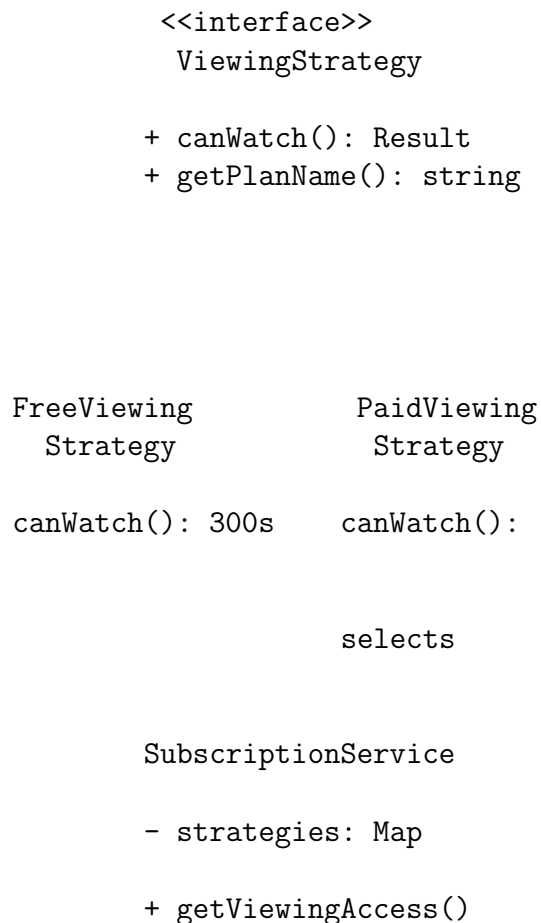
```
1 // src/modules/subscription/subscription.service.ts
2 import { Injectable } from '@nestjs/common';
3 import { SubscriptionPlan } from '@prisma/client';
4 import { ViewingStrategy } from '../strategies/viewing-strategy.
5   interface';
6 import { FreeViewingStrategy } from '../strategies/free-viewing.strategy';
7 import { PaidViewingStrategy } from '../strategies/paid-viewing.strategy';
8
9 @Injectable()
10 export class SubscriptionService {
11   private strategies: Record<SubscriptionPlan, ViewingStrategy>;
12
13   constructor(
14     private freeStrategy: FreeViewingStrategy,
15     private paidStrategy: PaidViewingStrategy,
16   ) {
17     this.strategies = {
18       [SubscriptionPlan.FREE]: this.freeStrategy,
19       [SubscriptionPlan.PAID]: this.paidStrategy,
20     };
21   }
22
23   getViewingAccess(plan: SubscriptionPlan, userId: string,
24     streamId: string) {
25     const strategy = this.strategies[plan];
26     return strategy.canWatch(userId, streamId);
27   }
28 }
```

```

26 }
27 }

```

6.3.3 Class Diagram



6.3.4 Why This Pattern Was Chosen

- **Open/Closed Principle:** New tiers (PREMIUM, ENTERPRISE) require only new strategy classes
- **Runtime Selection:** Algorithm chosen based on user's subscription plan
- **Testability:** Each strategy can be tested independently
- **Maintainability:** Viewing logic isolated from subscription logic
- **Flexibility:** Strategies can have different implementations (time limits, concurrent viewers, etc.)

6.4 Observer Pattern

6.4.1 Pattern Description

The Observer pattern defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified automatically.

6.4.2 Implementation: Stream Events

```
1 // src/modules/stream/stream.observer.ts
2 import { Injectable } from '@nestjs/common';
3 import { EventEmitter2 } from '@nestjs/event-emitter';
4
5 @Injectable()
6 export class StreamEventEmitter {
7   constructor(private eventEmitter: EventEmitter2) {}
8
9   emitStreamStarted(streamId: string, channelId: string,
10                    title: string) {
11     this.eventEmitter.emit('stream.started', {
12       streamId,
13       channelId,
14       title,
15       timestamp: new Date(),
16     });
17   }
18
19   emitStreamEnded(streamId: string) {
20     this.eventEmitter.emit('stream.ended', {
21       streamId,
22       timestamp: new Date(),
23     });
24   }
25 }

```

```
1 // src/modules/stream/stream.listener.ts
2 import { Injectable, Logger } from '@nestjs/common';
3 import { OnEvent } from '@nestjs/event-emitter';
4
5 @Injectable()
6 export class StreamEventListener {
7   private readonly logger = new Logger(StreamEventListener.name);
8
9   @OnEvent('stream.started')
10   handleStreamStarted(payload: any) {
11     this.logger.log('Stream started: ${payload.streamId} - ${payload.
12       title}');
13   }
14
15   @OnEvent('stream.ended')
16   handleStreamEnded(payload: any) {
17     this.logger.log('Stream ended: ${payload.streamId}');
18   }
19 }

```

```
1 // src/modules/chat/chat.gateway.ts
2 import { WebSocketGateway, WebSocketServer } from '@nestjs/websockets';

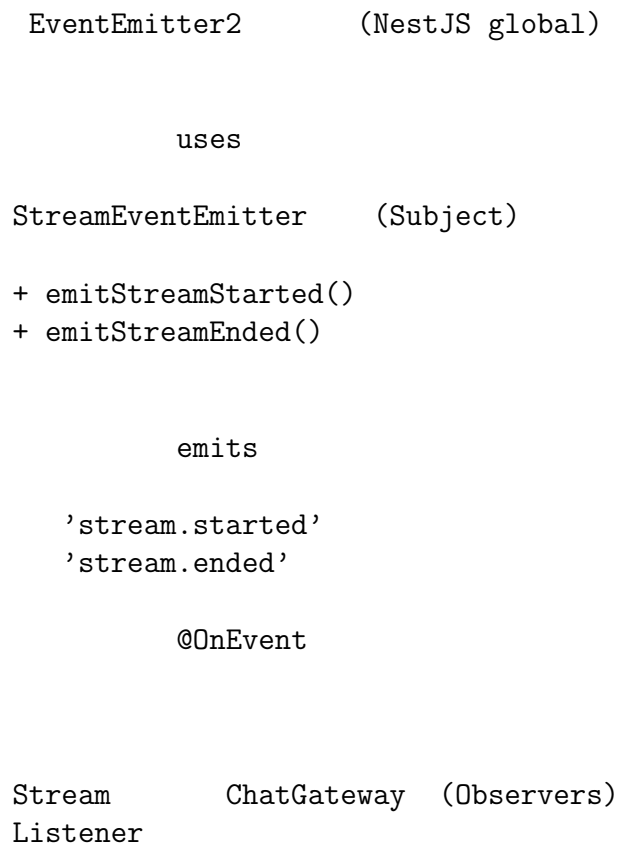
```

```

3 import { OnEvent } from '@nestjs/event-emitter';
4 import { Server } from 'socket.io';
5
6 @WebSocketGateway({ namespace: '/chat' })
7 export class ChatGateway {
8   @WebSocketServer()
9   server: Server;
10
11   @OnEvent('stream.started')
12   handleStreamStarted(payload: any) {
13     this.server.emit('streamStarted', {
14       streamId: payload.streamId,
15       channelId: payload.channelId,
16       title: payload.title,
17     });
18   }
19
20   @OnEvent('stream.ended')
21   handleStreamEnded(payload: any) {
22     this.server.emit('streamEnded', {
23       streamId: payload.streamId,
24       message: 'This stream has ended',
25     });
26   }
27 }

```

6.4.3 Class Diagram



```
@OnEvent      @OnEvent
handlers      + emit WS
```

6.4.4 Why This Pattern Was Chosen

- **Decoupling:** Stream module has zero knowledge of Chat module
- **Extensibility:** New observers (email notifications, analytics) require no changes to Stream
- **Asynchronous:** Event handling doesn't block stream operations
- **NestJS Integration:** Built-in EventEmitter2 support
- **Scalability:** Multiple listeners can react to same event

6.5 Adapter Pattern

6.5.1 Pattern Description

The Adapter pattern converts the interface of a class into another interface clients expect. It allows classes with incompatible interfaces to work together.

6.5.2 Implementation: Streaming Adapter

```
1 // src/modules/streaming/streaming.adapter.ts
2 export interface IStreamingAdapter {
3   getIngestUrl(streamKey: string): string;
4   getPlaybackUrl(streamKey: string): string;
5   validateStreamKey(streamKey: string): Promise<boolean>;
6 }
7
8 @Injectable()
9 export class NginxRtmpAdapter implements IStreamingAdapter {
10   private readonly rtmpHost = process.env.RTMP_HOST || 'localhost';
11   private readonly rtmpPort = process.env.RTMP_PORT || '1935';
12   private readonly hlsHost = process.env.HLS_HOST || 'localhost';
13   private readonly hlsPort = process.env.HLS_PORT || '8080';
14
15   getIngestUrl(streamKey: string): string {
16     return `rtmp://${this.rtmpHost}:${this.rtmpPort}/live/${streamKey}`;
17   }
18
19   getPlaybackUrl(streamKey: string): string {
20     return `http://${this.hlsHost}:${this.hlsPort}/hls/${streamKey}.m3u8`;
21   }
22
23   async validateStreamKey(streamKey: string): Promise<boolean> {
24     // Validation logic via database lookup
25     const channel = await this.channelRepository.findByStreamKey(
26       streamKey);
27     return !!channel;
```

```

27 }
28 }

```

6.5.3 Class Diagram

```

<<interface>>
IStreamingAdapter

+ getIngestUrl(): string
+ getPlaybackUrl(): str
+ validateStreamKey(): ok

```

implements

```

NginxRtmpAdapter

- rtmpHost: string
- hlsHost: string

+ getIngestUrl()
+ getPlaybackUrl()
+ validateStreamKey()

```

adapts

```

Nginx RTMP Server
(external infrastructure)

```

6.5.4 Why This Pattern Was Chosen

- **Infrastructure Abstraction:** Business logic doesn't depend on Nginx specifics
- **Switchability:** Can replace Nginx with AWS IVS, Wowza, etc. by implementing new adapter
- **Environment Configuration:** URL construction centralized
- **Testability:** Mock adapter for unit tests
- **Single Responsibility:** StreamingService focuses on orchestration, not URL construction

6.6 Template Method Pattern

6.6.1 Pattern Description

The Template Method pattern defines the skeleton of an algorithm in a base class but lets subclasses override specific steps without changing the algorithm's structure.

6.6.2 Implementation: BaseService

```

1 // src/common/base/base.service.ts
2 import { Injectable } from '@nestjs/common';
3 import { BaseRepository } from '../base.repository';
4
5 @Injectable()
6 export abstract class BaseService<T> {
7   constructor(protected repository: BaseRepository<T>) {}
8
9   // Template method - defines algorithm skeleton
10  async create(data: any): Promise<T> {
11    await this.beforeCreate(data);
12    const entity = await this.repository.create(data);
13    await this.afterCreate(entity);
14    return entity;
15  }
16
17  async update(id: string, data: any): Promise<T> {
18    await this.beforeUpdate(id, data);
19    const entity = await this.repository.update(id, data);
20    await this.afterUpdate(entity);
21    return entity;
22  }
23
24  async delete(id: string): Promise<void> {
25    await this.beforeDelete(id);
26    await this.repository.delete(id);
27    await this.afterDelete(id);
28  }
29
30  // Hook methods - can be overridden by subclasses
31  protected async beforeCreate(data: any): Promise<void> {}
32  protected async afterCreate(entity: T): Promise<void> {}
33  protected async beforeUpdate(id: string, data: any): Promise<void> {}
34  protected async afterUpdate(entity: T): Promise<void> {}
35  protected async beforeDelete(id: string): Promise<void> {}
36  protected async afterDelete(id: string): Promise<void> {}
37 }

```

```

1 // src/modules/channel/channel.service.ts
2 import { Injectable, Logger } from '@nestjs/common';
3 import { BaseService } from '../../common/base/base.service';
4 import { Channel } from '@prisma/client';
5 import { ChannelRepository } from '../channel.repository';
6
7 @Injectable()
8 export class ChannelService extends BaseService<Channel> {
9   private readonly logger = new Logger(ChannelService.name);
10 }

```



```

11  constructor(repository: ChannelRepository) {
12      super(repository);
13  }
14
15  // Override hook to add custom behavior
16  protected async afterCreate(channel: Channel): Promise<void> {
17      this.logger.log('Channel created: ${channel.name} (${channel.id})')
18      ;
19      // Could send welcome email, create analytics record, etc.
20  }
21
22  // Override hook for validation
23  protected async beforeDelete(id: string): Promise<void> {
24      const channel = await this.repository.findById(id);
25      if (!channel) {
26          throw new NotFoundException('Channel not found');
27      }
28      this.logger.warn('Deleting channel: ${channel.name}');
29  }

```

6.6.3 Class Diagram

BaseService<T>

repository: BaseRepository<T>

+ create(data): Promise<T>	← Template method
+ update(id, data): Promise<T>	← Template method
+ delete(id): Promise<void>	← Template method
# beforeCreate(data): void	← Hook (overridable)
# afterCreate(entity): void	← Hook (overridable)
# beforeUpdate(...): void	← Hook (overridable)
# afterUpdate(entity): void	← Hook (overridable)
# beforeDelete(id): void	← Hook (overridable)
# afterDelete(id): void	← Hook (overridable)

extends

ChannelSvc

UserSvc

afterCreate() (override)	beforeDel() (override)
-----------------------------	---------------------------

6.6.4 Why This Pattern Was Chosen

- **Code Reuse:** CRUD skeleton written once, used by all services
- **Consistency:** All services follow the same lifecycle (before/after hooks)
- **Extensibility:** Subclasses customize behavior without modifying base algorithm
- **Open/Closed:** Base class closed for modification, open for extension via hooks
- **Maintainability:** Cross-cutting concerns (logging, validation) in predictable locations

6.7 Composite Pattern

6.7.1 Pattern Description

The Composite pattern composes objects into tree structures to represent part-whole hierarchies. It lets clients treat individual objects and compositions uniformly.

6.7.2 Implementation: Channel Hierarchy

```
1 // src/modules/channel/channel.composite.ts
2 export interface ChannelComponent {
3   getDetails(): any;
4 }
5
6 // Leaf node
7 export class CommentLeaf implements ChannelComponent {
8   constructor(
9     private id: string,
10    private content: string,
11    private user: { name: string },
12    private createdAt: Date,
13  ) {}
14
15  getDetails() {
16    return {
17      id: this.id,
18      content: this.content,
19      user: this.user.name,
20      createdAt: this.createdAt,
21    };
22  }
23 }
24
25 // Branch node
26 export class StreamComposite implements ChannelComponent {
27   private comments: CommentLeaf[] = [];
28
29   constructor(
30     private id: string,
31     private title: string,
32     private status: string,
33   ) {}
34 }
```

```
35     addComment(comment: CommentLeaf): void {
36         this.comments.push(comment);
37     }
38
39     getDetails() {
40         return {
41             id: this.id,
42             title: this.title,
43             status: this.status,
44             comments: this.comments.map((c) => c.getDetails()),
45         };
46     }
47 }
48
49 // Root node
50 export class ChannelComposite implements ChannelComponent {
51     private streams: StreamComposite[] = [];
52
53     constructor(
54         private id: string,
55         private name: string,
56     ) {}
57
58     addStream(stream: StreamComposite): void {
59         this.streams.push(stream);
60     }
61
62     getDetails() {
63         return {
64             id: this.id,
65             name: this.name,
66             streams: this.streams.map((s) => s.getDetails()),
67         };
68     }
69 }
70
71 // Factory function to build composite tree
72 export function buildChannelComposite(data: any): ChannelComponent {
73     const channel = new ChannelComposite(data.id, data.name);
74
75     for (const streamData of data.streams || []) {
76         const stream = new StreamComposite(
77             streamData.id,
78             streamData.title,
79             streamData.status,
80         );
81
82         for (const commentData of streamData.comments || []) {
83             const comment = new CommentLeaf(
84                 commentData.id,
85                 commentData.content,
86                 commentData.user,
87                 commentData.createdAt,
88             );
89             stream.addComment(comment);
90         }
91
92         channel.addStream(stream);
93     }
94 }
```

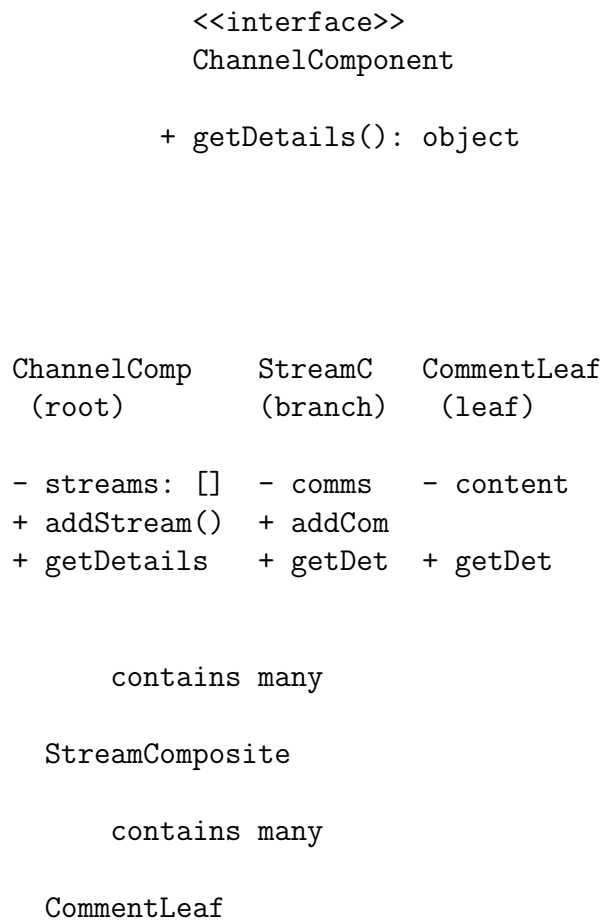
```

93   }
94
95   return channel;
96 }

1 // Usage in controller
2 @Get('/:id/hierarchy')
3 async getHierarchy(@Param('id') id: string) {
4   const data = await this.channelService.findWithStreamsAndComments(id)
5   ;
6   const composite = buildChannelComposite(data);
7   return composite.getDetails();
8 }

```

6.7.3 Class Diagram



6.7.4 Why This Pattern Was Chosen

- **Uniform Interface:** All components (channel, stream, comment) treated uniformly via `getDetails()`
- **Tree Structure:** Naturally represents Channel → Streams → Comments hierarchy
- **Recursive Composition:** `getDetails()` recursively traverses entire tree

- **Flexibility:** Easy to add new node types (e.g., StreamCategory)
- **Client Simplification:** Controller doesn't need conditional logic for different node types

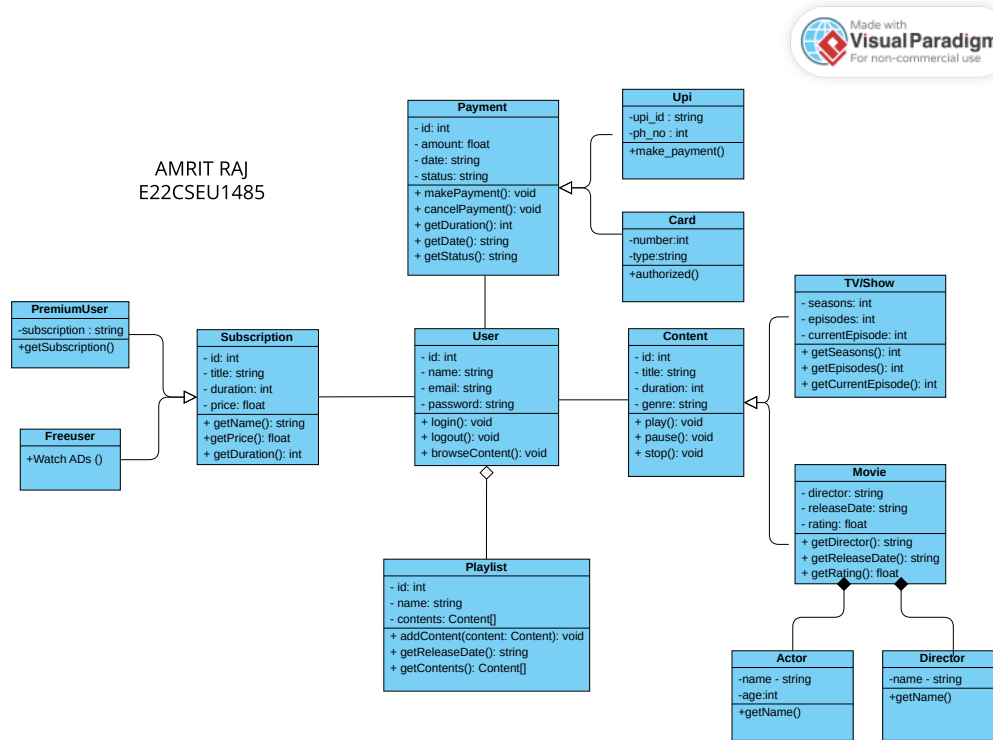


Figure 4: Class Diagram - Complete System Architecture

7 SOLID Principles Analysis

This section demonstrates how each SOLID principle is implemented throughout the codebase with concrete code-level evidence.

7.1 Single Responsibility Principle (SRP)

Principle Definition: A class should have only one reason to change.

7.1.1 Evidence in Architecture

The four-layer architecture enforces SRP:

```

1 // Controllers: ONLY handle HTTP concerns
2 @Controller('streams')
3 export class StreamController {
4     constructor(private streamService: StreamService) {}
5
6     @Post()
7     @UseGuards(JwtAuthGuard)
  
```

```

8   async create(@Body() dto: CreateStreamDto,
9       @CurrentUser() user: AuthenticatedUser) {
10      return this.streamService.create(dto, user.channelId);
11  }
12  // No business logic, no database access
13 }
14
15 // Services: ONLY handle business logic
16 @Injectable()
17 export class StreamService extends BaseService<Stream> {
18   async create(dto: CreateStreamDto, channelId: string) {
19     const streamData = this.factory.createStandard({
20       title: dto.title,
21       channelId
22     });
23     return this.repository.create(streamData);
24   }
25   // No HTTP handling, no direct Prisma usage
26 }
27
28 // Repositories: ONLY handle data access
29 @Injectable()
30 export class StreamRepository extends BaseRepository<Stream> {
31   async findLiveStreams() {
32     return this.prisma.stream.findMany({
33       where: { status: StreamStatus.LIVE },
34       include: { channel: true },
35     });
36   }
37   // No business logic, no HTTP awareness
38 }

```

7.1.2 Additional Examples

Event Emitter vs Listener:

- StreamEventEmitter: ONLY emits events
- StreamEventListener: ONLY logs events
- ChatGateway: ONLY broadcasts via WebSocket

Each class has a single, well-defined responsibility.

7.2 Open/Closed Principle (OCP)

Principle Definition: Software entities should be open for extension but closed for modification.

7.2.1 Evidence: Strategy Pattern

Adding a new subscription tier requires ZERO changes to existing code:

```

1  // NEW FILE: premium-viewing.strategy.ts
2  @Injectable()
3  export class PremiumViewingStrategy implements ViewingStrategy {

```

```

4   canWatch(userId: string, streamId: string): ViewingResult {
5       return {
6           canWatch: true,
7           maxDuration: null,
8           reason: 'Premium tier: unlimited + HD quality',
9       };
10  }
11
12  getPlanName(): string {
13      return 'PREMIUM';
14  }
15  }
16
17  // ONLY CHANGE: Add to enum in schema.prisma
18  enum SubscriptionPlan {
19      FREE
20      PAID
21      PREMIUM //      NEW
22  }
23
24  // ONLY CHANGE: Register in module
25  providers: [
26      FreeViewingStrategy,
27      PaidViewingStrategy,
28      PremiumViewingStrategy, //      NEW
29  ]

```

Existing strategies (Free, Paid) remain untouched. This is OCP in action.

7.2.2 Evidence: Observer Pattern

Adding email notifications requires ZERO changes to Stream module:

```

1  // NEW FILE: email-notification.listener.ts
2  @Injectable()
3  export class EmailNotificationListener {
4      @OnEvent('stream.started')
5      async sendStartedEmail(payload: any) {
6          await this.emailService.send({
7              to: payload.subscribers,
8              subject: `${payload.title} is now live!`,
9          });
10  }
11  }

```

StreamEventEmitter doesn't know or care about email notifications.

7.2.3 Evidence: Template Method

Custom behavior via hooks without modifying BaseService:

```

1  // BaseService.ts remains unchanged
2  // Subclasses extend via hooks:
3
4  export class UserService extends BaseService<User> {
5      protected async beforeCreate(data: any): Promise<void> {
6          data.password = await bcrypt.hash(data.password, 12);
7      }

```

```
8 }
9
10 export class ChannelService extends BaseService<Channel> {
11   protected async afterCreate(entity: Channel): Promise<void> {
12     await this.analytics.trackChannelCreation(entity.id);
13   }
14 }
```

7.3 Liskov Substitution Principle (LSP)

Principle Definition: Subtypes must be substitutable for their base types.

7.3.1 Evidence: BaseService Substitutability

All services extending BaseService can be used interchangeably:

```
1 function performCrud<T>(service: BaseService<T>, data: any) {
2   await service.create(data);
3   await service.update(id, updates);
4   await service.delete(id);
5   // Works identically for UserService, ChannelService, StreamService
6 }
```

7.3.2 Evidence: Strategy Pattern

All strategies return the same ViewingResult shape:

```
1 function checkAccess(strategy: ViewingStrategy): ViewingResult {
2   return strategy.canWatch(userId, streamId);
3   // Works for FreeViewingStrategy, PaidViewingStrategy, any future
4   // strategy
5 }
6
7 // Result structure is guaranteed:
8 interface ViewingResult {
9   canWatch: boolean;
10  maxDuration: number | null;
11  reason?: string;
12 }
```

7.3.3 Evidence: Composite Pattern

All composite components work uniformly:

```
1 function display(component: ChannelComponent) {
2   console.log(component.getDetails());
3   // Works for ChannelComposite, StreamComposite, CommentLeaf
4   // No need to check type - polymorphism handles it
5 }
```

7.4 Interface Segregation Principle (ISP)

Principle Definition: Clients should not be forced to depend on interfaces they don't use.

7.4.1 Evidence: Narrow Interfaces

Each interface defines only what's needed:

```
1 // ViewingStrategy: 2 methods ONLY
2 interface ViewingStrategy {
3     canWatch(userId: string, streamId: string): ViewingResult;
4     getPlanName(): string;
5     // No unrelated methods like billing, user management, etc.
6 }
7
8 // IStreamingAdapter: 3 methods ONLY
9 interface IStreamingAdapter {
10     getIngestUrl(streamKey: string): string;
11     getPlaybackUrl(streamKey: string): string;
12     validateStreamKey(streamKey: string): Promise<boolean>;
13     // No unrelated methods like transcoding config, analytics, etc.
14 }
15
16 // ChannelComponent: 1 method ONLY
17 interface ChannelComponent {
18     getDetails(): any;
19     // No CRUD methods, no HTTP handling, just tree traversal
20 }
```

7.4.2 Evidence: Separate DTOs

Controllers use minimal, purpose-specific DTOs:

```
1 // CREATE: Only title required
2 export class CreateStreamDto {
3     @IsString()
4     @NotEmpty()
5     title: string;
6 }
7
8 // UPDATE: Only title optional
9 export class UpdateStreamDto {
10     @IsString()
11     @Optional()
12     title?: string;
13 }
14
15 // Not a single bloated DTO with all fields
```

7.4.3 Evidence: Guards

Guards implement single-method interface:

```
1 interface CanActivate {
2     canActivate(context: ExecutionContext): boolean | Promise<boolean>;
3     // ONLY this method, nothing else
4 }
5
6 export class JwtAuthGuard implements CanActivate {
7     canActivate(context) { /* JWT validation only */ }
8 }
```

```

9
10 export class SubscriptionGuard implements CanActivate {
11     canActivate(context) { /* Subscription check only */ }
12 }

```

7.5 Dependency Inversion Principle (DIP)

Principle Definition: High-level modules should not depend on low-level modules. Both should depend on abstractions.

7.5.1 Evidence: NestJS Dependency Injection

All dependencies injected via constructor, no `new` keywords:

```

1 @Injectable()
2 export class StreamService {
3     constructor(
4         private repository: StreamRepository, // Abstraction
5         private factory: StreamFactory,       // Abstraction
6         private eventEmitter: StreamEventEmitter, // Abstraction
7     ) {}
8
9     // No manual instantiation:
10    //     this.repository = new StreamRepository()
11    //     DI container provides instance
12 }

```

7.5.2 Evidence: Repository Abstraction

Services depend on BaseRepository, not Prisma:

```

1 // BaseRepository abstracts data layer
2 export abstract class BaseRepository<T> {
3     constructor(protected prisma: PrismaService) {}
4
5     abstract create(data: any): Promise<T>;
6     abstract findById(id: string): Promise<T | null>;
7     abstract update(id: string, data: any): Promise<T>;
8     abstract delete(id: string): Promise<void>;
9 }
10
11 // Services depend on abstraction:
12 export class StreamService extends BaseService<Stream> {
13     constructor(protected repository: BaseRepository<Stream>) {
14         super(repository);
15         // Doesn't know it's Prisma underneath
16     }
17 }

```

Could swap Prisma for TypeORM/Mongoose without changing StreamService.

7.5.3 Evidence: Adapter Pattern

StreamingService depends on IStreamingAdapter interface:

```

1 @Injectable()
2 export class StreamingService {
3   constructor(
4     @Inject('STREAMING_ADAPTER')
5     private adapter: IStreamingAdapter // Interface, not concrete
6     class
7   ) {}
8
9   getUrls(streamKey: string) {
10     return {
11       ingest: this.adapter.getIngestUrl(streamKey),
12       playback: this.adapter.getPlaybackUrl(streamKey),
13     };
14   }
15
16   // Could provide different adapters:
17   providers: [
18     {
19       provide: 'STREAMING_ADAPTER',
20       useClass: NginxRtmpAdapter // or AwsIvsAdapter, WowzaAdapter
21     },
22   ]

```

High-level StreamingService doesn't depend on Nginx implementation.

8 API Design

8.1 RESTful Endpoints

The API follows RESTful conventions with resource-based URIs and standard HTTP methods.

Table 1: Authentication Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/register	Public	Register user, create channel, initialize FREE subscription
POST	/api/auth/login	Public	Login with email/password, returns JWT token

Table 2: Channel Endpoints

Method	Endpoint	Auth	Description
GET	/api/channels/me	JWT	Get authenticated user's channel with streamKey
GET	/api/channels/:id	Public	Get channel info (streamKey hidden)
GET	/api/channels/:id/hierarchy	Public	Composite tree: Channel→Streams→Comments

Table 3: Stream Endpoints

Method	Endpoint	Auth	Description
POST	/api/streams	JWT	Create new stream (PENDING status)
PATCH	/api/streams/:id/start	JWT	Start stream (LIVE status, emit event)
PATCH	/api/streams/:id/stop	JWT	End stream (ENDED status, emit event)
GET	/api/streams/live	Public	List all LIVE streams
GET	/api/streams/:id	Public	Get stream details
GET	/api/streams/channel/:channelId	Public	Get all streams for a channel

Table 4: Subscription Endpoints

Method	Endpoint	Auth	Description
POST	/api/subscriptions/subscribe	JWT	Upgrade to PAID (30-day expiry)
GET	/api/subscriptions/status	JWT	Get current plan and expiry
GET	/api/subscriptions/viewing-access	JWT	Get viewing rules via Strategy pattern

Table 5: Streaming Infrastructure Endpoints

Method	Endpoint	Auth	Description
GET	/api/streaming/urls	JWT	Get RTMP ingest and HLS playback URLs
POST	/api/streaming/auth	Public	Nginx on_publish callback (validates streamKey)

8.2 WebSocket Events

Real-time communication via Socket.io on namespace `/chat`:

Table 6: WebSocket Events

Event	Direction	Description
joinRoom	Client → Server	Join stream chat room with {streamId}
leaveRoom	Client → Server	Leave chat room
sendMessage	Client → Server	Send message: {streamId, userId, content}
existingMessages	Server → Client	Chat history (last 50 messages) on join
newMessage	Server → Client	Real-time broadcast: {id, content, user, createdAt}
streamStarted	Server → Client	Observer event: {streamId, channelId, title}
streamEnded	Server → Client	Observer event: {streamId, message}

8.3 Authentication Flow

JWT-based stateless authentication:

1. User sends credentials to POST /api/auth/login
2. Backend validates email/password via bcrypt
3. JWT token generated with payload: {userId, email, channelId}
4. Token returned to client (7-day expiry)
5. Client stores token (localStorage) and includes in Authorization header
6. Protected routes use JwtAuthGuard (Passport.js)
7. Guard validates token signature and expiry
8. Guard loads user from database via JwtStrategy
9. User object attached to request: req.user
10. Controllers access via @CurrentUser() decorator

```

1 // Client-side request
2 axios.get('/api/channels/me', {
3   headers: {
4     Authorization: 'Bearer ${token}'
5   }
6 });
7
8 // Server-side guard
9 @Get('me')
10 @UseGuards(JwtAuthGuard)
11 async getMyChannel(@CurrentUser() user: AuthenticatedUser) {
12   return this.channelService.findByUserId(user.userId);
13 }

```

8.4 Request/Response Examples

8.4.1 Register User

Request:

```
1 POST /api/auth/register
2 Content-Type: application/json
3
4 {
5   "name": "John Doe",
6   "email": "john@example.com",
7   "password": "SecurePass123",
8   "channelName": "JohnGaming"
9 }
```

Response:

```
1 {
2   "user": {
3     "id": "clx1a2b3c",
4     "name": "John Doe",
5     "email": "john@example.com"
6   },
7   "channel": {
8     "id": "clx4d5e6f",
9     "name": "JohnGaming"
10  },
11  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
12 }
```

8.4.2 Get Live Streams

Request:

```
1 GET /api/streams/live
```

Response:

```
1 [
2   {
3     "id": "clx7g8h9i",
4     "title": "Speedrun Marathon",
5     "status": "LIVE",
6     "startedAt": "2026-05-04T10:30:00Z",
7     "channel": {
8       "id": "clx4d5e6f",
9       "name": "JohnGaming"
10    }
11  }
12 ]
```

9 Frontend Architecture

9.1 Component Hierarchy

The React frontend follows a component-based architecture:

```
App (BrowserRouter)
  Navbar
    Logo
    Navigation Links
    Auth Status
  HomePage
    StreamGrid
      StreamCard (multiple)
  LoginPage
    LoginForm
  RegisterPage
    RegisterForm
  DashboardPage
    ChannelInfo
    StreamingConfig (OBS setup)
    GoLiveButton
  WatchPage
    VideoPlayer (HLS.js)
    StreamInfo
    ChatBox (Socket.io)
    FreeTimeTimer (conditional)
```

9.2 Routing

React Router v6 with protected routes:

```
1 // App.jsx
2 function App() {
3   return (
4     <BrowserRouter>
5       <Navbar />
6       <Routes>
7         <Route path="/" element={<HomePage />} />
8         <Route path="/login" element={<LoginPage />} />
9         <Route path="/register" element={<RegisterPage />} />
10        <Route path="/dashboard" element={
11          <ProtectedRoute>
12            <DashboardPage />
13          </ProtectedRoute>
14        } />
15        <Route path="/watch/:streamId" element={<WatchPage />} />
16      </Routes>
17    </BrowserRouter>
18  );
19 }
20
21 // ProtectedRoute component
22 function ProtectedRoute({ children }) {
23   const token = localStorage.getItem('token');
24   return token ? children : <Navigate to="/login" />;
25 }
```

9.3 State Management

The application uses local component state with React hooks:

- **useState:** Local UI state (form inputs, modals)
- **useEffect:** Side effects (API calls, WebSocket connection)
- **useContext:** Auth state (user, token)
- **Custom Hooks:** Reusable logic (useAuth, useSocket)

```
1 // useAuth hook
2 function useAuth() {
3   const [user, setUser] = useState(null);
4   const [token, setToken] = useState(localStorage.getItem('token'));
5
6   const login = async (email, password) => {
7     const { data } = await api.post('/auth/login', { email, password });
8     ;
9     setToken(data.token);
10    setUser(data.user);
11    localStorage.setItem('token', data.token);
12  };
13
14  const logout = () => {
15    setToken(null);
16    setUser(null);
17    localStorage.removeItem('token');
18  };
19
20  return { user, token, login, logout };
21 }
```

9.4 Video Player Integration

HLS.js integration for adaptive streaming:

```
1 // VideoPlayer.jsx
2 import Hls from 'hls.js';
3
4 function VideoPlayer({ streamKey, isLive }) {
5   const videoRef = useRef(null);
6   const hlsUrl = `http://localhost:8080/hls/${streamKey}.m3u8`;
7
8   useEffect(() => {
9     if (!Hls.isSupported()) {
10      // Fallback for browsers with native HLS support (Safari)
11      videoRef.current.src = hlsUrl;
12      return;
13    }
14
15    const hls = new Hls({
16      liveDurationInfinity: true,
17      liveBackBufferLength: 0,
18      enableWorker: true,
```



```
19   });
20
21   hls.loadSource(hlsUrl);
22   hls.attachMedia(videoRef.current);
23
24   hls.on(Hls.Events.MANIFEST_PARSED, () => {
25     videoRef.current.play();
26   });
27
28   return () => hls.destroy();
29 }, [streamKey]);
30
31 return (
32   <div className="video-container">
33     <video ref={videoRef} controls autoPlay />
34     {isLive && <span className="live-badger">LIVE</span>}
35   </div>
36 );
37 }
```

9.5 Real-Time Chat

Socket.io integration for live chat:

```
1 // ChatBox.jsx
2 import io from 'socket.io-client';
3
4 function ChatBox({ streamId, userId }) {
5   const [messages, setMessages] = useState([]);
6   const [socket, setSocket] = useState(null);
7
8   useEffect(() => {
9     const newSocket = io('http://localhost:3001/chat');
10    setSocket(newSocket);
11
12    newSocket.emit('joinRoom', { streamId });
13
14    newSocket.on('existingMessages', (msgs) => {
15      setMessages(msgs);
16    });
17
18    newSocket.on('newMessage', (msg) => {
19      setMessages((prev) => [...prev, msg]);
20    });
21
22    return () => newSocket.disconnect();
23  }, [streamId]);
24
25  const sendMessage = (content) => {
26    socket.emit('sendMessage', { streamId, userId, content });
27  };
28
29  return (
30    <div className="chat-box">
31      <div className="messages">
32        {messages.map((msg) => (
33          <div key={msg.id}>
```

```
34         <strong>{msg.user.name}:</strong> {msg.content}
35     </div>
36   )}}
37 </div>
38   <ChatInput onSend={sendMessage} />
39 </div>
40 );
41 }
```

10 Security Implementation

10.1 Authentication Mechanisms

10.1.1 Password Security

- **Hashing Algorithm:** bcrypt with 12 salt rounds
- **Salt:** Automatically generated per password
- **Storage:** Hashed password stored in database, plaintext never persisted
- **Comparison:** bcrypt.compare() for constant-time verification

```
1 // Registration
2 const hashedPassword = await bcrypt.hash(password, 12);
3 await this.userService.create({ ...userData, password: hashedPassword
4   });
5
6 // Login
7 const isValid = await bcrypt.compare(password, user.password);
8 if (!isValid) throw new UnauthorizedException('Invalid credentials');
```

10.1.2 JWT Token Security

- **Secret Key:** Stored in environment variable (JWT_SECRET)
- **Expiration:** 7 days (configurable)
- **Payload:** {userId, email, channelId, iat, exp}
- **Signature:** HMAC SHA256
- **Verification:** Passport.js JwtStrategy

```
1 // Token generation
2 const payload = {
3   userId: user.id,
4   email: user.email,
5   channelId: user.channel.id
6 };
7 const token = this.jwtService.sign(payload);
8
9 // Token validation (JwtStrategy)
```

```
10 async validate(payload: JwtPayload) {
11   const user = await this.userService.findById(payload.userId);
12   if (!user) throw new UnauthorizedException();
13   return user; // Attached to req.user
14 }
```

10.2 Authorization Mechanisms

10.2.1 JwtAuthGuard

Protects routes requiring authentication:

```
1 @UseGuards(JwtAuthGuard)
2 @Get('me')
3 async getMyChannel(@CurrentUser() user: AuthenticatedUser) {
4   // Only accessible with valid JWT token
5 }
```

10.2.2 SubscriptionGuard

Enforces premium features:

```
1 @Injectable()
2 export class SubscriptionGuard implements CanActivate {
3   async canActivate(context: ExecutionContext): Promise<boolean> {
4     const request = context.switchToHttp().getRequest();
5     const user = request.user;
6
7     const subscription = await this.subscriptionService.findById(
8       user.userId);
9
10    if (subscription.plan === SubscriptionPlan.FREE) {
11      throw new ForbiddenException('Premium feature - upgrade required');
12    }
13
14    if (subscription.expiresAt && subscription.expiresAt < new Date()) {
15      throw new ForbiddenException('Subscription expired');
16    }
17
18    return true;
19  }
20
21  // Usage
22  @UseGuards(JwtAuthGuard, SubscriptionGuard)
23  @Get('premium-feature')
24  async accessPremium() {
25    // Only accessible to PAID users with active subscription
26  }
```

10.3 Input Validation

NestJS ValidationPipe with class-validator:

```

1 // DTO with validation
2 export class RegisterDto {
3   @IsString()
4   @IsNotEmpty()
5   name: string;
6
7   @IsEmail()
8   email: string;
9
10  @IsString()
11  @MinLength(8)
12  @Matches(/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)/)
13  password: string;
14
15  @IsString()
16  @IsNotEmpty()
17  channelName: string;
18 }
19
20 // Global pipe in main.ts
21 app.useGlobalPipes(new ValidationPipe({
22   whitelist: true,           // Strip unknown properties
23   forbidNonWhitelisted: true, // Throw error on extra properties
24   transform: true,          // Auto-transform to DTO type
25 }));

```

Validation Features:

- Type checking (string, number, boolean)
- Format validation (email, URL, UUID)
- Length constraints (min/max)
- Pattern matching (regex)
- Custom validators
- Automatic sanitization

10.4 RTMP Authentication

Nginx RTMP on_publish callback:

```

1 application live {
2   live on;
3   on_publish http://localhost:3001/api/streaming/auth;
4 }

```

```

1 // Backend validation
2 @Post('auth')
3 async validateStreamKey(@Body() body: { name: string }) {
4   const streamKey = body.name; // Nginx sends stream key as 'name'
5
6   const channel = await this.channelRepository.findByStreamKey(
7     streamKey);

```

```

8   if (!channel) {
9       throw new UnauthorizedException('Invalid stream key');
10  }
11
12  return { success: true }; // Nginx accepts stream
13 }

```

Security Flow:

1. OBS sends RTMP stream with stream key in URL
2. Nginx extracts stream key from URL
3. Nginx POSTs to backend: {name: streamKey}
4. Backend queries database for matching channel
5. If found: return 200 OK → Nginx accepts stream
6. If not found: return 401 → Nginx rejects stream
7. Invalid streams never enter the system

10.5 CORS Configuration

Cross-Origin Resource Sharing protection:

```

1 // main.ts
2 app.enableCors({
3   origin: process.env.FRONTEND_URL || 'http://localhost:5173',
4   credentials: true,
5   methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
6   allowedHeaders: ['Content-Type', 'Authorization'],
7 });

```

Restricts API access to frontend origin only.

11 Streaming Pipeline

11.1 RTMP to HLS Conversion

The streaming infrastructure follows this pipeline:

```

OBS Studio (RTMP encoder)
  ↓ rtmp://localhost:1935/live/{streamKey}
Nginx RTMP Server (port 1935)
  ↓ on_publish callback → validate stream key
  ↓ ffmpeg transcoding
HLS Segments (.ts) + Playlist (.m3u8)
  ↓ written to /tmp/hls/
Nginx HTTP Server (port 8080)
  ↓ http://localhost:8080/hls/{streamKey}.m3u8
Frontend (HLS.js)
  ↓ adaptive bitrate playback
Browser Video Element

```

11.2 Nginx Configuration

Complete nginx.conf:

```
1 worker_processes auto;
2 events {
3     worker_connections 1024;
4 }
5
6 rtmp {
7     server {
8         listen 1935;
9         chunk_size 4096;
10
11         application live {
12             live on;
13             record off;
14
15             # HLS output
16             hls on;
17             hls_path /tmp/hls;
18             hls_fragment 3s;
19             hls_playlist_length 30s;
20
21             # Authentication callback
22             on_publish http://localhost:3001/api/streaming/auth;
23             on_done http://localhost:3001/api/streaming/done;
24         }
25     }
26 }
27
28 http {
29     server {
30         listen 8080;
31
32         location /hls {
33             types {
34                 application/vnd.apple.mpegurl m3u8;
35                 video/mp2t ts;
36             }
37             root /tmp;
38
39             # CORS headers for browser playback
40             add_header Cache-Control no-cache;
41             add_header Access-Control-Allow-Origin *;
42         }
43     }
44 }
```

Configuration Details:

- **hls_fragment 3s:** Each .ts segment is 3 seconds long
- **hls_playlist_length 30s:** Playlist contains 10 segments (30s / 3s)
- **on_publish:** Validates stream key before accepting
- **CORS headers:** Allows browser playback from any origin
- **chunk_size 4096:** Optimized for low-latency streaming

11.3 OBS Setup

Broadcaster configuration:

1. Install OBS Studio
2. Navigate to Settings → Stream
3. Service: Custom
4. Server: `rtmp://localhost:1935/live`
5. Stream Key: (from Dashboard API call to `/api/channels/me`)
6. Settings → Output:
 - Encoder: x264
 - Bitrate: 2500 kbps
 - Keyframe Interval: 2
 - Preset: veryfast
7. Click "Start Streaming"
8. Backend receives `on_publish` callback
9. If valid: stream appears at `http://localhost:8080/hls/{streamKey}.m3u8`

11.4 HLS Playback

Frontend playback flow:

1. User navigates to `/watch/:streamId`
2. Frontend fetches stream details from `GET /api/streams/:id`
3. Extract `streamKey` from channel data
4. Construct HLS URL: `http://localhost:8080/hls/{streamKey}.m3u8`
5. HLS.js loads `.m3u8` playlist
6. Parser extracts `.ts` segment URLs
7. Sequential `.ts` segment downloads
8. Media Source Extensions (MSE) feeds video element
9. Adaptive bitrate: HLS.js monitors bandwidth, switches quality
10. 3-5 second latency from broadcaster to viewer

11.5 Latency Optimization

Achieved latency: 3-5 seconds glass-to-glass

Optimization Techniques:

- Short segment duration (3s reduces initial buffering)
- Small playlist window (30s = 10 segments)
- HLS.js low-latency mode: `liveDurationInfinity: true`
- Zero back-buffer: `liveBackBufferLength: 0`
- Aggressive chunking: `chunk_size 4096`
- No transcoding (single quality) in MVP

Future Improvements:

- LL-HLS (Low-Latency HLS) with partial segments
- WebRTC for sub-second latency
- Adaptive bitrate encoding (multiple qualities)
- CDN integration for global distribution

12 Testing & Deployment

12.1 Setup Instructions

12.1.1 Prerequisites

- Node.js v18+ and npm
- PostgreSQL database or NeonDB account
- Nginx with RTMP module
- OBS Studio (for streaming)

12.1.2 Backend Setup

```
1 # Clone repository
2 cd server
3
4 # Install dependencies
5 npm install
6
7 # Configure environment
8 cp .env.example .env
9 # Edit .env:
10 # DATABASE_URL="postgresql://user:pass@host:5432/dbname"
11 # JWT_SECRET="your-secret-key"
12
```



```
13 # Generate Prisma client
14 npx prisma generate
15
16 # Run migrations
17 npx prisma migrate dev
18
19 # Start development server
20 npm run start:dev # Runs on http://localhost:3001
```

12.1.3 Frontend Setup

```
1 cd frontend
2
3 # Install dependencies
4 npm install
5
6 # Configure environment
7 # Edit src/config.js with backend URL
8
9 # Start development server
10 npm run dev # Runs on http://localhost:5173
```

12.1.4 Nginx Setup

```
1 # Install Nginx with RTMP module
2 # Ubuntu/Debian:
3 sudo apt install nginx libnginx-mod-rtmp
4
5 # Copy configuration
6 sudo cp nginx/nginx.conf /etc/nginx/nginx.conf
7
8 # Create HLS directory
9 sudo mkdir -p /tmp/hls
10 sudo chmod 777 /tmp/hls
11
12 # Test configuration
13 sudo nginx -t
14
15 # Restart Nginx
16 sudo systemctl restart nginx
```

12.2 Running the Application

Terminal 1 - Backend:

```
1 cd server
2 npm run start:dev
3 # Backend running on http://localhost:3001
```

Terminal 2 - Frontend:

```
1 cd frontend
2 npm run dev
3 # Frontend running on http://localhost:5173
```

Terminal 3 - Nginx:

```
1 sudo nginx
2 # RTMP on port 1935
3 # HLS HTTP on port 8080
```

12.3 Testing the Streaming Flow

1. Register User

- Navigate to `http://localhost:5173/register`
- Create account with channel name
- Auto-login after registration

2. Get Stream Key

- Navigate to Dashboard
- Copy RTMP URL and Stream Key

3. Configure OBS

- Paste RTMP URL: `rtmp://localhost:1935/live`
- Paste Stream Key from Dashboard

4. Create Stream

- Click "Create Stream" on Dashboard
- Enter stream title

5. Go Live

- Click "Start Streaming" in OBS
- Click "Go Live" button in Dashboard
- Stream status changes to LIVE

6. Watch Stream

- Navigate to Home page
- See live stream in grid
- Click to watch
- Video player loads HLS stream
- Chat is functional

12.4 Testing Patterns

12.4.1 Strategy Pattern Test

```
1 # Test FREE tier (5-minute limit)
2 GET /api/subscriptions/viewing-access
3 Authorization: Bearer {token}
4
5 Response: { canWatch: true, maxDuration: 300 }
6
7 # Upgrade to PAID
8 POST /api/subscriptions/subscribe
9 Authorization: Bearer {token}
10
11 # Test PAID tier (unlimited)
12 GET /api/subscriptions/viewing-access
13 Authorization: Bearer {token}
14
15 Response: { canWatch: true, maxDuration: null }
```

12.4.2 Observer Pattern Test

```
1 # Terminal 1: Watch backend logs
2 npm run start:dev
3
4 # Terminal 2: Connect WebSocket client
5 wscat -c ws://localhost:3001/chat
6 > {"event": "joinRoom", "data": {"streamId": "..."}}
7
8 # Terminal 3: Start stream
9 PATCH /api/streams/:id/start
10
11 # Observe:
12 # - Terminal 1: StreamEventListener logs "Stream started"
13 # - Terminal 2: ChatGateway emits "streamStarted" event
```

13 UML Diagram Explanations

13.1 Sequence Diagram (sequence.png)

The sequence diagram illustrates the complete request flow through the system layers.

Key Interactions Shown:

1. User Authentication Flow:

- User submits login credentials to Frontend
- Frontend sends POST request to AuthController
- AuthController delegates to AuthService
- AuthService validates credentials via UserRepository
- UserRepository queries PrismaService
- PrismaService executes database query

- JWT token generated and returned through layers
- Frontend stores token for subsequent requests

2. Protected Resource Access:

- User requests protected resource with JWT token
- JwtAuthGuard intercepts request
- Guard validates token via JwtStrategy
- JwtStrategy loads user from database
- User object attached to request
- Controller receives authenticated request
- Service processes business logic
- Repository fetches data
- Response flows back through layers

3. Stream Lifecycle:

- User creates stream via StreamController
- StreamService uses StreamFactory for object creation
- StreamRepository persists to database
- User starts stream (PATCH /streams/:id/start)
- StreamService updates status to LIVE
- StreamEventEmitter emits 'stream.started' event
- StreamEventListener logs event
- ChatGateway broadcasts to WebSocket clients
- Multiple observers react without coupling

Design Patterns Visible:

- Layered architecture separation
- Singleton pattern (PrismaService single instance)
- Observer pattern (event emission to multiple listeners)
- Dependency injection (constructor parameters)

13.2 Use Case Diagram (usecase.png)

The use case diagram maps actors to system functionalities.

Actors:

1. Guest User (Unauthenticated):

- Register account
- Login

- View live streams
- Watch streams (5-minute limit)
- View channel information

2. **Authenticated User:**

- All Guest capabilities
- Create streams
- Start/stop streams
- Send chat messages
- Access dashboard
- View streaming URLs
- Manage channel

3. **Paid Subscriber:**

- All Authenticated capabilities
- Unlimited viewing duration
- Access premium features

4. **Nginx RTMP Server:**

- Validate stream key
- Ingest RTMP stream
- Transcode to HLS
- Serve HLS segments

Key Use Cases:

- **Streaming Workflow:** Create Stream → Configure OBS → Go Live → Viewers Watch → End Stream
- **Subscription Upgrade:** View Status → Subscribe → Access Premium Features
- **Real-time Chat:** Join Room → Send Messages → Receive Messages
- **Authentication:** Register → Login → Access Protected Resources

Relationships:

- **<<include>>:** Mandatory dependencies (e.g., "Watch Stream" includes "Load HLS Playlist")
- **<<extend>>:** Optional extensions (e.g., "Send Chat" extends "Watch Stream")
- **<<inherit>>:** Actor inheritance (Authenticated User inherits Guest capabilities)

13.3 Class Diagram (class.png)

The class diagram shows the complete object-oriented structure with all design patterns.

Key Components:

1. Controllers Layer:

- AuthController, StreamController, ChannelController, etc.
- Depend on Service layer
- Use Guards (@UseGuards decorator)
- Define HTTP routes

2. Services Layer:

- BaseService`IT` (Template Method pattern)
- UserService, StreamService, ChannelService (extend BaseService)
- Depend on Repository layer
- Implement business logic

3. Repository Layer:

- BaseRepository`IT` (abstraction)
- UserRepository, StreamRepository, etc. (concrete implementations)
- Depend on PrismaService
- Isolate data access

4. Data Layer:

- PrismaService (Singleton pattern)
- Extends PrismaClient
- Manages database connection

5. Pattern Implementations:

- **Factory:** StreamFactory with createStandard(), createLive(), createScheduled()
- **Strategy:** ViewingStrategy interface → FreeViewingStrategy, PaidViewingStrategy
- **Observer:** StreamEventEmitter → StreamEventListener, ChatGateway (listeners)
- **Adapter:** IStreamingAdapter interface → NginxRtmpAdapter
- **Composite:** ChannelComponent → ChannelComposite, StreamComposite, CommentLeaf

6. Supporting Classes:

- Guards: JwtAuthGuard, SubscriptionGuard
- DTOs: CreateStreamDto, UpdateStreamDto, RegisterDto, LoginDto
- Interfaces: ViewingResult, JwtPayload, AuthenticatedUser

Relationships Shown:

- **Inheritance:** BaseService $\leftarrow$ UserService, StreamService
- **Implementation:** ViewingStrategy $\leftarrow$ FreeViewingStrategy
- **Association:** StreamController $\rightarrow$ StreamService
- **Dependency:** StreamService $\rightarrow$ StreamRepository
- **Composition:** ChannelComposite $\rightarrow$ StreamComposite
- **Aggregation:** StreamService $\rightarrow$ StreamFactory

13.4 Entity-Relationship Diagram (er.png)

The ER diagram models the database schema with entities, attributes, and relationships.

Entities:

1. User

- Primary Key: id (cuid)
- Attributes: name, email (unique), password (hashed), createdAt, updatedAt
- Relationships: 1:1 with Channel, 1:1 with Subscription, 1:N with Comment

2. Channel

- Primary Key: id (cuid)
- Attributes: name (unique), streamKey (unique, auto-generated), userId (FK), createdAt, updatedAt
- Relationships: N:1 with User, 1:N with Stream

3. Stream

- Primary Key: id (cuid)
- Attributes: title, status (enum: PENDING—LIVE—ENDED), startedAt (nullable), endedAt (nullable), channelId (FK), createdAt, updatedAt
- Relationships: N:1 with Channel, 1:N with Comment

4. Subscription

- Primary Key: id (cuid)
- Attributes: plan (enum: FREE—PAID), expiresAt (nullable), userId (FK), createdAt, updatedAt

- Relationships: N:1 with User (1:1 enforced via unique constraint)

5. Comment

- Primary Key: id (cuid)
- Attributes: content, userId (FK), streamId (FK), createdAt
- Relationships: N:1 with User, N:1 with Stream

Cardinality:

- User (1) (1) Channel
- User (1) (1) Subscription
- User (1) (N) Comment
- Channel (1) (N) Stream
- Stream (1) (N) Comment

Constraints:

- Unique constraints: email, channel.name, channel.streamKey
- Cascade deletions: Deleting User cascades to Channel, Subscription, Comments
- Indexes: email, streamKey, channelId, status, streamId, userId
- Enums: StreamStatus, SubscriptionPlan (type safety)

Design Decisions Visible:

- Normalized schema (3NF) - no redundant data
- Foreign key integrity ensures referential consistency
- Nullable timestamps (startedAt, endedAt) support state transitions
- Enum types enforce valid values at database level
- Strategic indexing optimizes common queries (live streams, channel lookup)

14 Team Contribution Matrix

Table 7: Team Member Contributions

Member	Responsibilities	Contribution %
Ranajeet Roy	Project Lead & Architecture - System design and architecture planning - Implementation of all 7 Gang-of-Four design patterns - SOLID principles application across codebase - BaseService and BaseRepository abstractions - Strategy pattern (viewing strategies) - Observer pattern (event system) - Template Method pattern (service lifecycle) - Composite pattern (channel hierarchy) - Factory pattern (stream creation) - Adapter pattern (streaming infrastructure) - Singleton pattern (PrismaService) - Code review and quality assurance - UML diagram creation (sequence, class, ER) - Technical documentation and report writing	30%
Narendra Singh	Backend Development & Streaming Infrastructure - NestJS backend architecture setup - Nginx RTMP server configuration - RTMP to HLS transcoding pipeline - OBS integration and stream validation - Streaming module implementation - RTMP authentication callback system - HLS segment generation and serving - Stream lifecycle management (start/stop) - Real-time event handling for stream status - Performance optimization for low-latency streaming	25%
Pranav Singh	Backend Development & API Design - REST API endpoint implementation - Authentication system (JWT, bcrypt) - Authorization guards and middleware - Database schema design with Prisma - Repository layer implementation - Input validation with class-validator - WebSocket gateway for real-time chat - Subscription management system - Channel and user management modules - API testing and debugging	25%
Jivit Rana	Frontend Development & UI/UX - React application architecture - Component hierarchy design - HLS.js video player integration - Real-time chat interface with Socket.io - User authentication flows (login/register) - Dashboard and streaming controls UI - Responsive design implementation - State management with React hooks - UML diagrams (use case, activities)	10%

Collaboration Highlights:

- **Daily Standups:** Team met daily to discuss progress and blockers
- **Code Reviews:** All pull requests reviewed by Ranajeet (project lead)
- **Pair Programming:** Backend team (Narendra & Pranav) paired on complex modules
- **Integration Testing:** Frontend and backend teams collaborated on API contract testing
- **Documentation:** All members contributed to inline code documentation
- **Git Workflow:** Feature branch workflow with protected main branch

15 Conclusion

15.1 Summary

This project successfully demonstrates the practical application of advanced software engineering principles in a production-grade live streaming platform. We implemented a full-stack application that handles real-time video streaming, WebSocket-based chat, subscription management, and secure authentication.

15.2 Achievements

Technical Accomplishments:

1. **Design Patterns:** Successfully implemented all seven Gang-of-Four patterns with clear justifications and measurable benefits
2. **SOLID Principles:** Achieved strict adherence to all five principles throughout the codebase
3. **Scalable Architecture:** Four-layer architecture separates concerns and enables independent scaling
4. **Real-time Capabilities:** Low-latency streaming (3-5 seconds) and instant chat messaging
5. **Security:** Comprehensive security model with JWT authentication, bcrypt hashing, and input validation
6. **Production-Ready:** Professional codebase with TypeScript type safety, error handling, and logging

Learning Outcomes:

1. Deep understanding of when and why to apply specific design patterns
2. Hands-on experience with modern full-stack technologies (NestJS, React, Prisma, Nginx)

3. Practical knowledge of video streaming protocols (RTMP, HLS)
4. Real-world application of software architecture principles
5. Team collaboration in a complex, multi-module project

15.3 Challenges Overcome

- **Nginx RTMP Integration:** Configuring Nginx with RTMP module and implementing authentication callback
- **HLS Latency:** Optimizing segment duration and playlist length for low-latency streaming
- **WebSocket State Management:** Synchronizing stream status across multiple connected clients
- **Pattern Selection:** Choosing appropriate patterns without over-engineering
- **Database Design:** Modeling relationships to support both CRUD operations and complex queries
- **Type Safety:** Maintaining strict TypeScript types across frontend and backend

15.4 Future Improvements

Short-term Enhancements:

1. **Additional Subscription Tiers:** Implement PREMIUM and ENTERPRISE plans with Strategy pattern
2. **Stream Analytics:** Track viewer count, peak concurrent users, watch time
3. **Email Notifications:** Observer pattern extension for email alerts (stream started, subscription expiring)
4. **Moderation Tools:** Chat moderation, user blocking, timeout functionality
5. **Stream Categories:** Categorization and tagging system for content discovery

Long-term Scalability:

1. **CDN Integration:** CloudFront or Cloudflare for global HLS delivery
2. **Multi-quality Streaming:** Adaptive bitrate encoding (360p, 720p, 1080p)
3. **WebRTC Integration:** Sub-second latency for interactive streams
4. **Horizontal Scaling:** Redis for session storage, Kafka for event streaming
5. **Microservices:** Split into independent services (Auth, Streaming, Chat)
6. **Kubernetes Deployment:** Container orchestration for auto-scaling
7. **Machine Learning:** Content recommendation engine based on viewing history

Additional Features:

1. VOD (Video on Demand): Recording and playback of past streams
2. Stream Scheduling: Calendar-based scheduling with reminders
3. Donations/Monetization: Payment integration for viewer support
4. Mobile Apps: Native iOS and Android applications
5. Collaborative Streaming: Multi-host streams with co-streaming

15.5 Reflections

This project provided invaluable experience in:

- Translating theoretical software design concepts into production code
- Balancing design pattern usage with pragmatic simplicity
- Understanding trade-offs in architecture decisions
- Collaborating effectively in a team with distributed responsibilities
- Writing maintainable, well-documented code for future developers

The most significant lesson learned was that design patterns are not prescriptive solutions to be forced into code, but rather tools to be applied judiciously when they solve real problems and improve code quality.

16 References

References

- [1] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [2] Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- [3] Ozer, J. (2020). *Video Streaming 101: Understanding Protocols and Codecs*. Streaming Media Magazine.
- [4] NestJS Documentation. (2026). *Official NestJS Framework Documentation*. Retrieved from <https://docs.nestjs.com>
- [5] Prisma Documentation. (2026). *Prisma ORM Documentation*. Retrieved from <https://www.prisma.io/docs>
- [6] Agentzh. (2024). *nginx-rtmp-module Documentation*. GitHub Repository.
- [7] Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT) RFC 7519*. Internet Engineering Task Force.

- [8] Pantos, R., & May, W. (2017). *HTTP Live Streaming RFC 8216*. Internet Engineering Task Force.
- [9] Fette, I., & Melnikov, A. (2011). *The WebSocket Protocol RFC 6455*. Internet Engineering Task Force.